Lesson 2: Smooth movement

Pygame CE • S3 onwards • 50-60 min

Learning intention

Use the game loop to make movement continuous while a key is held.

Success criteria

- I can use `pygame.key.get_pressed()`.
- I can explain what speed does.
- I can explain why `dt` is used.

Key ideas

- **get_pressed()** reports which keys are held *right now*, every frame.
- **dt** is the time (in seconds) since the last frame. Speed x dt keeps movement consistent on fast and slow computers.
- Positions are now decimal numbers, so convert with `int()` when drawing.
- A true value counts as 1 and false as 0, so `keys[RIGHT] - keys[LEFT]` gives -1, 0 or 1.

Activities

1. Compare Lesson 1 movement with the smooth version.
2. Try speeds of 100, 250 and 600. Describe the effect.
3. Change the frame rate in `clock.tick()` and observe that speed stays similar.

Code

Key changes

```
dt = clock.tick(60) / 1000
keys = pygame.key.get_pressed()
speed = 250

x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * speed * dt
y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) * speed * dt
```

Complete program

```
import pygame

pygame.init()

WIDTH = 800
HEIGHT = 600
SIZE = 40

screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Smooth movement")
clock = pygame.time.Clock()

x = 380.0
y = 280.0
speed = 250
running = True

while running:
    dt = clock.tick(60) / 1000

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
```

```
            running = False

    keys = pygame.key.get_pressed()
    x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * speed * dt
    y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) * speed * dt

    screen.fill((30, 30, 40))
    pygame.draw.rect(screen, (80, 200, 120), (int(x), int(y), SIZE, SIZE))
    pygame.display.flip()

pygame.quit()
```

Checkpoint

The shape moves smoothly while a key is held. Learners can explain how speed and dt affect movement.

Support and stretch

- **Support:** Start with horizontal movement only.
- **Stretch:** Add a sprint key (hold Shift to go faster). Challenge: stop diagonal movement being faster than straight movement.

Reflect

Why is the old `clock.tick(60)` at the bottom of the loop removed?

Save your work
Save this version as `lesson2.py` before you start the next lesson.